

System Architecture & Technical Handoff Document

Project: Justice-Chain Sentinel (justice_chain)
Target AI / Role: Lead Technical Architect & Senior Flutter Engineer
Document Version: 1.3

1. Executive Summary & Vision

Justice-Chain Sentinel is an edge-native, zero-trust personal safety and cryptographic evidence preservation platform built in Flutter. The platform is engineered to operate under extreme emergency conditions, including physical distress, user incapacitation, host device destruction, or total network blackouts.

Core Engineering Principles

- **Zero-Trust & Serverless:** No central user database or authorization server. All identity credentials and keypairs are generated and held strictly on the local device keystore.
- **Reactive Edge Execution:** Hardware monitoring, camera controls, and local database writes are orchestrated synchronously using reactive signals without UI context dependency.
- **Cryptographic Tamper-Proofing:** Raw captured evidence is hashed, encrypted, anchored to decentralized storage (IPFS), and anchored to an immutable blockchain ledger to guarantee an undeniable chain of custody.

2. Technology Stack & Dependencies

Category	Package / Tool	Version / Standard	Functional Purpose
Framework	Flutter / Dart	SDK ^3.9.2	Core cross-platform engine
State Management	signals_flutter	^6.3.1	Fine-grained, reactive state tracking & background effects
Local Storage	hive_flutter	^1.1.0	High-speed local database for vault logs & guardian whitelists
Hardware Keystore	flutter_secure_storage	^11.2.0	Secure storage for master keys and Node ID credentials
Cryptography	crypto, encrypt	AES-256-CBC, SHA-256	Cryptographic hashing, file encryption, and stream digests
Hardware Controls	camera, record	Resolution: Medium	Video evidence capture & audio monitoring
Decentralized Storage	http, dio, Pinata API	IPFS Gateway	Asynchronous encrypted payload upload and remote retrieval
P2P Mesh Networking	nearby_connections	Google Nearby API	Offline device discovery and direct data routing
Optical Identity	qr_flutter, mobile_scanner	QR Code Protocol	Off-grid public key exchange and guardian pairing
Blockchain Bridge	web3dart / Local RPC	Hardhat / Anvil / Fabric	Smart contract invocation and ledger verification

3. Directory & File Structure Reference

```
lib/
|-- core/
|   |-- ai_service.dart           # Audio processing & ML inference bindings
|   |-- app_services.dart        # Global service locators & logger setup
|   |-- audio_preprocessor.dart   # PCM stream transformation
|   |-- crypto_utils.dart         # AES-256 file encryption/decryption (EvidenceEncryptor)
|   |-- emergency_controller.dart # Camera session & record pipeline controller
|   |-- evidence_retriever.dart   # IPFS fetch & local decryption pipeline
|   |-- identity_service.dart     # Keystore manager & Ed25519 identity generator
|   |-- mesh_service.dart         # Google Nearby Connections P2P transport
|   |-- pinata_service.dart       # Multipart IPFS upload bridge via Pinata API
|-- data/                         # Local models and Hive adapters
|-- logic/
|   |-- safety_signals.dart       # Reactive signals (appStatus, isRecording, etc.)
`-- presentation/
    |-- guardian_pairing_screen.dart # QR generation for local Node ID share
    |-- guardian_scanner_screen.dart # QR scanner for guardian whitelist addition
    |-- home_screen.dart           # Main Sentinel operational dashboard
    |-- registration_screen.dart   # First-time local identity setup
main.dart                         # Application bootstrap & initialization
```

4. Module Implementation Status & Architectural Flow

Module A: Decentralized Identity & Local Keystore

Implementation Status: COMPLETE

Architectural Flow:

- 1 IdentityService generates a unique cryptographic keypair on device initialization.
- 2 The private key is committed directly into the hardware Keystore (flutter_secure_storage).
- 3 The raw public key is hashed using **SHA-256**, extracting a unique 16-character string designated as the device's **Node ID**.
- 4 User authentication (PIN) is salted and hashed via SHA-256 before local persistence in Hive. Plaintext keys or PINs never touch the application log or disk.

Module B: Optical Guardian Pairing & Whitelisting

Implementation Status: PARTIALLY COMPLETE (Pairing operational, Mesh Transfer pending)

Architectural Flow:

- 1 GuardianPairingScreen packages the user's Node ID and public key into a compact JSON payload, displayed visually using qr_flutter.
- 2 GuardianScannerScreen utilizes mobile_scanner to read a prospective guardian's QR payload offline.
- 3 Decoded identity data is written to the local Hive database (vault_box) as a trusted contact whitelist entry.

Current Limitation: Optical pairing and whitelist registration are fully functional. Direct video payload broadcast/transfer over nearby_connections to a guardian node during an internet blackout is **untested and pending implementation**.

Module C: Reactive AI Listening & Emergency Capture

Implementation Status: **COMPLETE**

Architectural Flow:

- 1 `aiDistressDetected`, `cameraReady`, and `isRecording` are tracked synchronously via global reactive signals (`signals_flutter`).
- 2 An `effect()` reactor inside `MainSafetyScreen` listens for trigger conditions (`aiDistressDetected == true`).
- 3 Upon trigger, the system invokes `EmergencyController.startRecording()` asynchronously via `unawaited()`, preserving UI responsiveness.
- 4 A hardcoded **45-second failsafe timer** guarantees that recording automatically halts and commits evidence even if the user is rendered unable to interact with the device.

Module D: Cryptographic Evidence Sealing & Vault Ledger

Implementation Status: **COMPLETE**

Architectural Flow:

- 1 Video capture runs strictly at `ResolutionPreset.medium` to avoid exceeding memory bounds or hardware bus constraints during emergency operations.
- 2 Immediately upon session termination, the raw `.mp4` file is read as a binary stream and hashed via `sha256.bind()`.
- 3 The system creates a new entry in `Hive.box('vault_box')` logging the local path, SHA-256 hash, ISO-8601 timestamp, and initial status (`locally_secured`).

Module E: Encryption, IPFS Upload & Storage Pipeline

Implementation Status: **IN PROGRESS** (Encryption & IPFS Upload working; Purging pending)

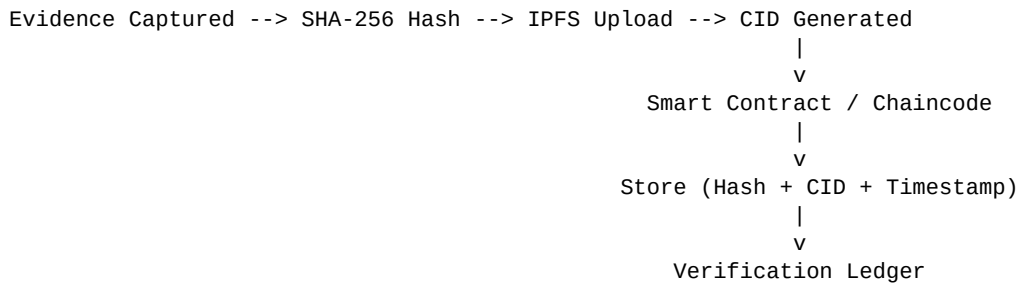
Architectural Flow:

- 1 Captured raw video is encrypted locally via `EvidenceEncryptor.encryptVideoFile()` (`lib/core/crypto_utils.dart`) using **AES-256-CBC**.
- 2 The symmetric key is committed to `flutter_secure_storage` tagged by the generated filename.
- 3 `PinataService.uploadToIPFS()` transmits the encrypted payload (`.enc`) to decentralized storage via Pinata's API.
- 4 Once uploaded, the returned **IPFS Content Identifier (CID)** is written back to the Hive Vault record, updating its status to `encrypted_and_uploaded`.

Current Limitation: The encrypted payload is successfully generated and uploaded to IPFS. However, **automatic deletion of the raw `.mp4` and temporary `.enc` files from local device storage is currently incomplete/unverified**. Complete zero-persistence local data purging must be finalized.

5. Blockchain Integration Strategy (Two-Phase Architecture)

To ensure rapid delivery of the core evidence preservation pipeline without infrastructure delays, the blockchain layer is divided into two execution phases.



Phase 1: Blockchain Simulation / Local EVM Environment

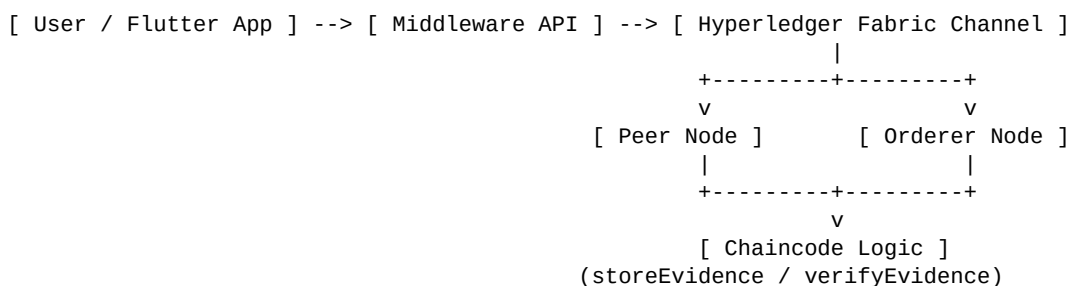
Initially, the blockchain anchoring layer will run on a **local Ethereum-compatible environment** (e.g., Hardhat or Anvil).

- **Objective:** Validate end-to-end cryptographic anchoring and contract interaction without needing real cryptocurrency or testnet faucet tokens.
- **Smart Contract Interface (Solidity):**
 - `storeEvidence(string sha256Hash, string ipfsCID, uint256 timestamp, string nodeId)`
 - `verifyEvidence(string sha256Hash)` returns `(bool, string ipfsCID, uint256 timestamp)`
 - `getEvidenceHistory(string nodeId)` returns `(EvidenceRecord[])`
- **App Integration:** The Flutter app uses web3dart over HTTP RPC to send transactions directly to the local node upon successful IPFS upload.

Phase 2: Private Consortium Ledger via Hyperledger Fabric

Once the primary workflow is proven, the local EVM environment will be upgraded to a **permissioned, enterprise-grade Hyperledger Fabric network**.

- **Objective:** Ensure evidence is governed exclusively by authorized jurisdictional entities rather than a public network.
- **Network Consortium Participants:**
 - 1 **Law Enforcement / Police:** Submits and verifies chain-of-custody requests.
 - 2 **Forensic Laboratories:** Validates raw media hashes against stored ledger entries.
 - 3 **Judicial Courts:** Queries ledger for tamper-proof evidentiary proof.
 - 4 **Justice-Chain Organization Node:** Coordinates identity lookup and network maintenance.



- **Access Control:** Each organization possesses its own cryptographic certificate issued via Fabric's Certificate Authority (CA).

- **Chaincode Functions:** Matches the functional interface defined in Phase 1 (`storeEvidence`, `verifyEvidence`, `getEvidenceHistory`), executed within private channels.

6. Immediate Action Plan & Development Objectives

When taking over development, tackle these objectives in order of structural priority:

Priority 1: Finalize Zero-Persistence File Purging

Ensure complete teardown of local evidence bytes once IPFS upload is confirmed:

- Implement explicit file deletion for both the raw source `.mp4` file and the intermediate `.enc` file inside `_uploadToIpfsInBackground()`.
- Add startup garbage collection to purge lingering media files in temporary directories on application launch.

Priority 2: Implement Mesh Video Transfer (Nearby Connections)

Build out `mesh_service.dart` to support off-grid evidence migration:

- Establish P2P payload routing using `nearby_connections`.
- Implement a receiver pipeline on guardian nodes to identify incoming encrypted video payloads, verify sender Node IDs against the local Hive whitelist, and save the payload to disk.

Priority 3: Guardian Decryption & Playback UI

Build out authorized media viewing screens:

- Integrate `EvidenceRetriever.fetchAndDecryptFromIpfs()` into a user-facing playback interface.
- Implement key-exchange handling so whitelisted guardians can retrieve the corresponding AES-256 key needed to decrypt and view the remote IPFS stream.

Priority 4: Implement Phase 1 Blockchain Anchoring (Local EVM)

- Write and deploy the `EvidenceLedger.sol` contract to a local Hardhat/Anvil instance.
- Wire the Flutter app (`web3dart`) to automatically call `storeEvidence()` upon receiving an IPFS CID from Pinata.

Priority 5: Prepare Phase 2 Hyperledger Fabric Architecture

- Set up Docker containers for Fabric CA, orderer nodes, and peer nodes for the consortium organizations.
- Write equivalent Go/TypeScript chaincode and build a REST gateway for backend-to-Fabric communication.